IoT Based Smart Irrigation Monitoring System Using Raspberry Pi

1. Introduction

Agriculture requires efficient water management to improve crop productivity and reduce water waste. Farmers often irrigate crops without knowing the exact soil moisture condition or upcoming weather changes.

This project proposes an **IoT-based smart irrigation monitoring system** that measures soil moisture and provides irrigation recommendations through a mobile application.

The system uses a **soil moisture sensor connected to a Raspberry Pi 4**, which sends real-time data to a cloud database using **Firebase**. A mobile application built using **MIT App Inventor** retrieves this data and combines it with weather information from **OpenWeather** to inform farmers when irrigation is required.

2. Objectives

The main objectives of the project are:

- Monitor soil moisture using sensors
- Send real-time sensor data to the cloud
- Display farm data in a mobile application
- Integrate weather information for irrigation advice
- Help farmers make better irrigation decisions

3. System Architecture

The system follows an IoT architecture where sensors collect environmental data and transmit it to a mobile application through a cloud platform.

None

Soil Moisture Sensor

```
    ↓
 Raspberry Pi
    ↓
Firebase Cloud Database
    ↓
 Mobile Application
    ↓
  Weather API
    ↓
Watering Recommendation
```

4. Hardware Components

Component	Purpose
Soil Moisture Sensor	Measures soil water level
Raspberry Pi 4	Processes sensor data
MCP3008 ADC	Converts analog sensor data to digital
Breadboard	Circuit connections
Jumper wires	Electrical connections
WiFi module	Internet communication

5. Software Components

Software	Purpose
Python	Raspberry Pi programming
Firebase	Cloud database
MIT App Inventor	Mobile app development
OpenWeather	Weather API service

6. Raspberry Pi Sensor Data Collection

The soil moisture sensor produces analog signals. Since the Raspberry Pi cannot read analog signals directly, an MCP3008 ADC converter is used.

The Raspberry Pi reads the sensor value and uploads it to the Firebase cloud database.

7. Raspberry Pi Python Code

Python

```
import spidev
import time
import requests

# Firebase database URL
firebase_url =
"https://smart-farm-app-2026-default-rtdb.asia-southeast1.firebas
edatabase.app/sensor_data.json"

# Initialize SPI communication
spi = spidev.SpiDev()
spi.open(0, 0)
spi.max_speed_hz = 1350000

# Function to read analog value
def read_channel(channel):
    adc = spi.xfer2([1, (8+channel)<<4, 0])
    data = ((adc[1] & 3) << 8) + adc[2]
    return data

while True:
    moisture_value = read_channel(0)
    print("Soil Moisture:", moisture_value)

    data = {"moisture": moisture_value}
    requests.put(firebase_url, json=data)
```

```
    time.sleep(10)
```

This program performs the following tasks:

1. Reads soil moisture sensor value
2. Converts analog signal to digital data
3. Sends the data to Firebase database
4. Repeats the process every 10 seconds

8. Firebase Cloud Database

The sensor data is stored in **Firebase Realtime Database**.

Example database structure:

```
None
smartfarm
  soil_moisture: 42
  temperature: 30
  weather: sunny
  crop: chilli
```

Firebase acts as the communication bridge between the Raspberry Pi and the mobile application.

9. Weather API Integration

Weather data is retrieved using the **OpenWeather API**.

Example API request:

```
None
https://api.openweathermap.org/data/2.5/weather?q=Coimbatore&appid=API_KEY&units=metric
```

This API provides:

- Temperature
- Humidity
- Weather conditions
- Rain probability

10. Mobile Application

The mobile application is developed using **MIT App Inventor**, a block-based visual programming platform suitable for rapid mobile app development.

Mobile App Features

- Crop selection (Tomato, Chilli, Rice, Groundnut, Sugarcane)
- Real-time soil moisture monitoring
- Weather information display
- Irrigation recommendation
- Moisture history graph
- Low moisture alert notification

App Screens

Screen 1 — Crop Selection: The farmer selects the crop being cultivated. The selection is stored in Firebase for use in recommendation logic.

Screen 2 — Dashboard: Displays real-time data retrieved from Firebase and OpenWeather.

```
None
Crop        : Chilli
Soil Moisture: 38%
Temperature  : 30°C
Weather      : Cloudy

Recommendation:
Water the plant today
```

Screen 3 — Moisture History: Displays a line graph of soil moisture readings over time, showing daily and weekly trends.

MIT App Inventor Components Used

- Labels — display sensor values
- Buttons — crop selection and navigation
- FirebaseDB — cloud data retrieval
- Web component — OpenWeather API calls
- Chart extension — moisture history graph
- Notifier component — low moisture alerts
- Clock component — auto-refresh every 10 seconds

11. Decision Logic for Irrigation

The system provides irrigation advice based on soil moisture and weather conditions.

```
None
If soil_moisture < crop_minimum
AND rain probability < 40%
→ Water the plant

If soil_moisture < crop_minimum
AND rain probability > 60%
→ Wait for rain

If soil_moisture is within range
→ Moisture is sufficient
```

Crop Moisture Thresholds

Crop	Min Moisture (%)	Max Moisture (%)
Tomato	40	60
Chilli	35	55
Rice	70	90
Groundnut	30	50

12. Advantages

- Efficient water usage
- Real-time soil monitoring
- Remote farm monitoring via mobile app
- Integration of weather information
- Simple interface suitable for farmers with no technical background

13. Limitations

- Requires internet connectivity
- Sensor calibration required for accuracy
- Firebase security rules must be configured properly
- Weather API requests may have usage limits

14. Future Enhancements

- Automatic irrigation pump control
- Machine learning based irrigation prediction
- Multi-sensor monitoring across large farms
- SMS alert system for farmers

15. Conclusion

The proposed IoT-based smart irrigation system demonstrates how sensor technology, cloud computing, and mobile applications can improve agricultural water management. By integrating soil moisture monitoring with weather data and delivering recommendations through a simple MIT App Inventor mobile application, the system helps farmers make informed irrigation decisions and optimize crop growth.

The mobile application was successfully built and tested, with Firebase and OpenWeather API fully integrated and working in real time.